

Laboratorio reto: Biblioteca binaria con préstamos

C++20 — sin solución guiada

1. Reglas

- La interfaz será un `main()` corto con llamadas de prueba.
- No uses bases de datos, JSON, CSV ni serialización externa.
- No reescribas un archivo completo para modificar stock, contadores o estados.
- No uses `while (!file.eof())`.
- Los datos deben sobrevivir al cierre y a una segunda ejecución.
- Toda lectura parcial de un registro iniciado debe tratarse como error de formato o archivo truncado.

2. Escenario y formato de archivos

Todos los archivos viven dentro de `library/`, que se crea automáticamente.

2.1 `codes.bin`

```
[nextBookCode] [nextLoanCode]
```

- Al crearse por primera vez, ambos valores comienzan en 1.
- Si el archivo ya existe, no se reinicializa.
- Solicitar un código devuelve el valor actual y deja almacenado el siguiente.

2.2 `books.bin`

```
[int code]
[uint32_t titleLength]
[titleLength bytes de titulo]
[int stock]
[float price]
[int loanCounter]
```

El título no incluye terminador nulo. `loanCounter` representa la cantidad histórica de préstamos.

2.3 `loans.bin`

```
[int loanCode]
[int bookCode]
[uint32_t borrowerLength]
[borrowerLength bytes del nombre]
[char active]
```

`active = 1`: préstamo vigente. `active = 0`: préstamo devuelto.

Las longitudes almacenadas en archivo son `std::uint32_t`. Los tamaños internos de memoria pueden usar `std::size_t`.

3. Requisitos funcionales

3.1 Inicialización

- Crear la carpeta y los tres archivos si faltan.
- Abrirlos para lectura y escritura binaria sin truncar datos existentes.
- Inicializar únicamente `codes.bin` cuando acaba de crearse.
- Ante una falla esencial de preparación, detener la construcción del objeto mediante una excepción.

3.2 Registrar libro

- Rechazar título vacío, stock negativo o precio negativo.
- Obtener un código persistente.
- Crear un registro completo según el formato de `books.bin`.
- Agregarlo al final sin alterar registros previos.
- Usar `tellp()` antes y después de escribir para comprobar que la cantidad de bytes escrita coincide con el tamaño calculado.

3.3 Listar libros

- Recorrer desde el inicio y mostrar código, título, stock, precio y contador.
- La condición del ciclo debe depender del éxito de leer el primer campo del siguiente registro.
- Si comienza un registro y falta algún campo, reportar fallo.

3.4 Prestar libro

- Buscar el libro por código mediante recorrido secuencial.
- Si no existe o su stock es cero, no generar préstamo ni consumir código.
- Disminuir stock e incrementar `loanCounter` directamente en sus posiciones dentro de `books.bin`.
- Agregar un préstamo activo al final de `loans.bin`.
- Usar `tellg()` para capturar las posiciones de los campos que luego serán modificados con `seekp()`.

3.5 Devolver préstamo

- Buscar el préstamo por código.
- Si no existe o ya está inactivo, devolver fallo sin modificar nada.
- Cambiar el estado a inactivo en su posición exacta.
- Buscar el libro asociado e incrementar su stock en su posición exacta.
- No disminuir `loanCounter`.

4. Restricciones de implementación

- Puedes usar una clase principal y una clase **Buffer**, pero la estructura final es decisión tuya.
- Los strings se almacenan como longitud seguida de bytes de caracteres.
- La serialización puede realizarse con un búfer o con escrituras directas; si usas **Buffer**, evita accesos desalineados.
- Usa de manera significativa **seekg**, **seekp**, **tellg** y **tellp**.
- Después de llegar a EOF o a un fallo, limpia las banderas antes de volver a posicionar el stream.
- Distingue entre “no encontrado” y “lectura incompleta” cuando corresponda.

5. Prueba mínima

Ejecuta estas pruebas en orden:

1. Registrar “Dune” con stock 2 y “1984” con stock 1.
2. Listar libros.
3. Prestar Dune a Ana y luego a Luis.
4. Intentar un tercer préstamo de Dune a Carlos; debe fallar por stock.
5. Devolver el primer préstamo.
6. Intentar devolver el mismo préstamo otra vez; debe fallar.
7. Listar libros nuevamente.
8. Cerrar el programa, ejecutar de nuevo sin registrar otra vez los libros y comprobar persistencia y continuidad de códigos.

Momento	Stock Dune	loanCounter	Resultado
Inicial	2	0	Libro disponible
Tras préstamo 1	1	1	Préstamo activo
Tras préstamo 2	0	2	Préstamo activo
Préstamo 3	0	2	Falla sin crear registro
Tras devolución 1	1	2	Préstamo inactivo
Devolución repetida	1	2	Falla sin cambios